

PRACTICAL-1

❖ **AIM:** Write a Java program to display “Hello World” and demonstrate the Java program structure and demonstrate use of variables, data types, and type casting.

❖ **CODE:**

```
public class Practical1 {
```

```
    public static void main(String[] args) {
```

```
        // Display Hello World
```

```
        System.out.println("Hello World");
```

```
        // Variables and Data Types
```

```
        byte age = 20;
```

```
        short year = 2026;
```

```
        int rollNo = 101;
```

```
        long mobile = 9876543210L;
```

```
        float marks = 89.5f;
```

```
        double salary = 35000.75;
```

```
        char grade = 'A';
```

```
        boolean result = true;
```

```
        // Display Variables
```

```
        System.out.println("\n--- Variables and Data Types ---");
```

```
        System.out.println("Age = " + age);
```

```
        System.out.println("Year = " + year);
```

```
        System.out.println("Roll Number = " + rollNo);
```

```
        System.out.println("Mobile = " + mobile);
```

```
        System.out.println("Marks = " + marks);
```

```
        System.out.println("Salary = " + salary);
```

```
        System.out.println("Grade = " + grade);
```

```
        System.out.println("Result = " + result);
```

```
        // Implicit Type Casting
```

```
        int num = 100;
```

```
        double d = num; // Automatic Conversion
```

```
        System.out.println("\n--- Implicit Type Casting ---");
```

```
        System.out.println("Integer Value = " + num);
```

```
        System.out.println("Converted Double Value = " + d);
```

// Explicit Type Casting

```
double pi = 3.14159;  
int value = (int) pi; // Manual Conversion  
  
System.out.println("\n--- Explicit Type Casting ---");  
System.out.println("Double Value = " + pi);  
System.out.println("Converted Integer Value = " + value);  
}  
}
```

❖ OUTPUT

Output Clear

```
Hello World  
  
--- Variables and Data Types ---  
Age = 20  
Year = 2026  
Roll Number = 101  
Mobile = 9876543210  
Marks = 89.5  
Salary = 35000.75  
Grade = A  
Result = true  
  
--- Implicit Type Casting ---  
Integer Value = 100  
Converted Double Value = 100.0  
  
--- Explicit Type Casting ---  
Double Value = 3.14159  
Converted Integer Value = 3  
  
=== Code Execution Successful ===
```

PRACTICAL-2

- ❖ **AIM:** Write a Java program to perform arithmetic, relational, and logical operations, bitwise and other operators.

❖ **CODE**

```
public class Practical2 {  
    public static void main(String[] args) {  
  
        int a = 10, b = 5;  
        boolean x = true, y = false;  
  
        // Arithmetic Operators  
        System.out.println("Arithmetic Operators");  
        System.out.println("a + b = " + (a + b));  
        System.out.println("a - b = " + (a - b));  
        System.out.println("a * b = " + (a * b));  
        System.out.println("a / b = " + (a / b));  
  
        // Relational Operators  
        System.out.println("\nRelational Operators");  
        System.out.println("a > b = " + (a > b));  
        System.out.println("a == b = " + (a == b));  
        System.out.println("a != b = " + (a != b));  
  
        // Logical Operators  
        System.out.println("\nLogical Operators");  
        System.out.println("x && y = " + (x && y));  
        System.out.println("x || y = " + (x || y));  
        System.out.println("!x = " + (!x));  
  
        // Bitwise Operators  
        System.out.println("\nBitwise Operators");  
        System.out.println("a & b = " + (a & b));  
        System.out.println("a | b = " + (a | b));  
        System.out.println("a ^ b = " + (a ^ b));  
    }  
}
```

// Assignment Operator

```
a += b;  
System.out.println("\nAssignment Operator");  
System.out.println("a += b = " + a);  
}  
}
```

❖ OUTPUT

Output Clear

```
Arithmetic Operators  
a + b = 15  
a - b = 5  
a * b = 50  
a / b = 2  
  
Relational Operators  
a > b = true  
a == b = false  
a != b = true  
  
Logical Operators  
x && y = false  
x || y = true  
!x = false  
  
Bitwise Operators  
a & b = 0  
a | b = 15  
a ^ b = 15  
  
Assignment Operator  
a += b = 15  
  
=== Code Execution Successful ===
```

PRACTICAL-3a

- ❖ **AIM:** Write a Java program using below conditional statements. a) Even or Odd using if.. else statement.

❖ **CODE**

```
import java.util.Scanner;

public class Practical3_a {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a number: ");
        int n = sc.nextInt();

        if (n % 2 == 0)
            System.out.println(n + " is Even");
        else
            System.out.println(n + " is Odd");
    }
}
```

❖ **OUTPUT**

Output Clear

```
Enter a number: 25
25 is Odd

=== Code Execution Successful ===
```

PRACTICAL-3b

❖ **AIM:** Roots of Quadratic Equation using else if ladder.

❖ **CODE**

```
import java.util.Scanner;

public class Practical3_b {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a, b, c: ");
        double a = sc.nextDouble();
        double b = sc.nextDouble();
        double c = sc.nextDouble();

        double d = b * b - 4 * a * c;

        if (d > 0) {
            double r1 = (-b + Math.sqrt(d)) / (2 * a);
            double r2 = (-b - Math.sqrt(d)) / (2 * a);
            System.out.println("Roots are: " + r1 + " and " + r2);
        } else if (d == 0) {
            double r = -b / (2 * a);
            System.out.println("Equal Roots: " + r);
        } else {
            System.out.println("Imaginary Roots");
        }
    }
}
```

❖ OUTPUT

```
Output Clear  
Enter a, b, c: 1  
-5  
1  
Roots are: 4.7912878474779195 and 0.20871215252208009  
  
=== Code Execution Successful ===
```

PRACTICAL-3C

❖ **AIM:** Largest of three numbers using nested if else.

❖ **CODE**

```
import java.util.Scanner;

public class Practical3_c {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter three numbers: ");
        int a = sc.nextInt();
        int b = sc.nextInt();
        int c = sc.nextInt();

        if (a > b) {
            if (a > c)
                System.out.println(a + " is Largest");
            else
                System.out.println(c + " is Largest");
        } else {
            if (b > c)
                System.out.println(b + " is Largest");
            else
                System.out.println(c + " is Largest");
        }
    }
}
```

❖ **OUTPUT**

OutputClear

```
Enter three numbers: 56
85
47
85 is Largest

=== Code Execution Successful ===
```


PRACTICAL-3d

❖ **AIM:** Find out the week day using switch statement.

❖ **CODE**

```
import java.util.Scanner;
public class Practical3_d {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter day number (1-7): ");
        int day = sc.nextInt();

        switch (day) {
            case 1:
                System.out.println("Sunday");
                break;
            case 2:
                System.out.println("Monday");
                break;
            case 3:
                System.out.println("Tuesday");
                break;
            case 4:
                System.out.println("Wednesday");
                break;
            case 5:
                System.out.println("Thursday");
                break;
            case 6:
                System.out.println("Friday");
                break;
            case 7:
                System.out.println("Saturday");
                break;
            default:
                System.out.println("Invalid Day");
                break;
        }
    }
}
```

❖ OUTPUT

```
Output Clear  
Enter day number (1-7): 7  
Saturday  
=== Code Execution Successful ===
```

PRACTICAL-4a

- ❖ **AIM:** Write a Java program to demonstrate looping constructs: a) Reverse of a number using while loop.

❖ **CODE**

```
import java.util.Scanner;

public class Practical4_a {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a number: ");
        int num = sc.nextInt();

        int reverse = 0;

        while (num != 0) {
            int digit = num % 10;
            reverse = reverse * 10 + digit;
            num = num / 10;
        }

        System.out.println("Reverse = " + reverse);
    }
}
```

❖ **OUTPUT**

Output Clear

```
Enter a number: 4598
Reverse = 8954

=== Code Execution Successful ===
```

PRACTICAL-4b

❖ **AIM:** Prime number using do while loop.

❖ **CODE**

```
import java.util.Scanner;

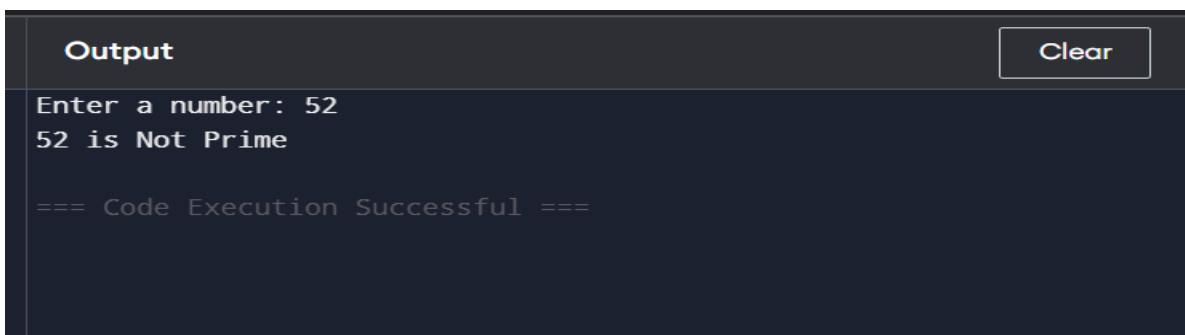
public class Practical4_b {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();

        int i = 2;
        boolean prime = true;

        do {
            if (n % i == 0 && n != 2) {
                prime = false;
                break;
            }
            i++;
        } while (i < n);

        if (prime && n > 1)
            System.out.println(n + " is Prime");
        else
            System.out.println(n + " is Not Prime");
    }
}
```

❖ **OUTPUT**



```
Output
Enter a number: 52
52 is Not Prime

=== Code Execution Successful ===
```

PRACTICAL-4c

❖ **AIM:** nth term of Fibonacci sequence using for loop.

❖ **CODE**

```
import java.util.Scanner;

public class Practical4_c {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of terms: ");
        int n = sc.nextInt();
        int a = 0, b = 1, c;

        System.out.print("Fibonacci Series: ");

        for (int i = 1; i <= n; i++) {
            System.out.print(a + " ");
            c = a + b;
            a = b;
            b = c;
        }
    }
}
```

❖ **OUTPUT**

Output	Clear
Enter number of terms: 10 Fibonacci Series: 0 1 1 2 3 5 8 13 21 34 === Code Execution Successful ===	

PRACTICAL-5

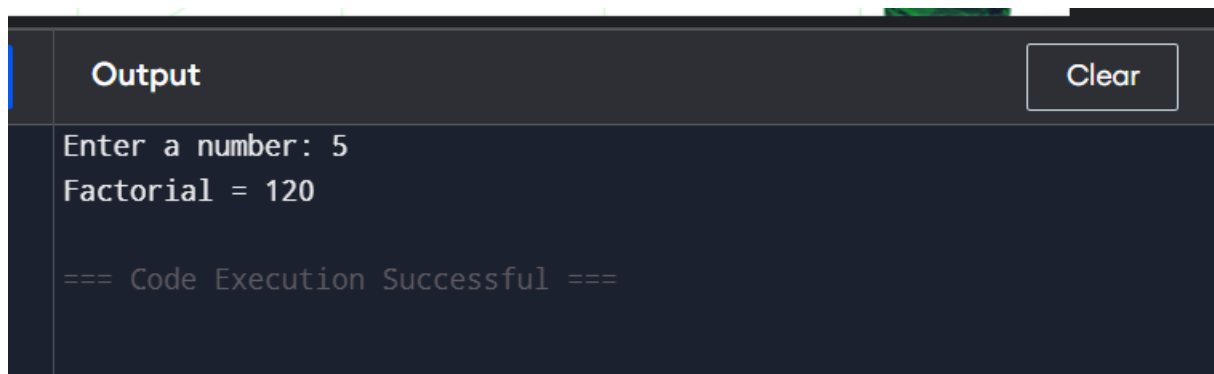
❖ **AIM:** Write a Java program to perform methods and example on recursion.

❖ **CODE**

```
import java.util.Scanner;

public class Practical5 {
    // Recursive Method
    static int factorial(int n) {
        if (n == 0 || n == 1)
            return 1;
        else
            return n * factorial(n - 1);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in)
        System.out.print("Enter a number: ");
        int num = sc.nextInt();
        System.out.println("Factorial = " + factorial(num));
    }
}
```

❖ **OUTPUT**



The screenshot shows a dark-themed IDE output window. At the top, there is a tab labeled 'Output' and a 'Clear' button. The output text is as follows:

```
Enter a number: 5
Factorial = 120

=== Code Execution Successful ===
```

PRACTICAL-6

❖ **AIM:** Write a Java program to perform operations on one-dimensional and multi-dimensional arrays.

❖ **CODE**

```
import java.util.Scanner;

public class Practical6 {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // One-Dimensional Array
        int[] arr = new int[5];
        int sum = 0;
        System.out.println("Enter 5 elements:");
        for (int i = 0; i < 5; i++) {
            arr[i] = sc.nextInt();
            sum += arr[i];
        }

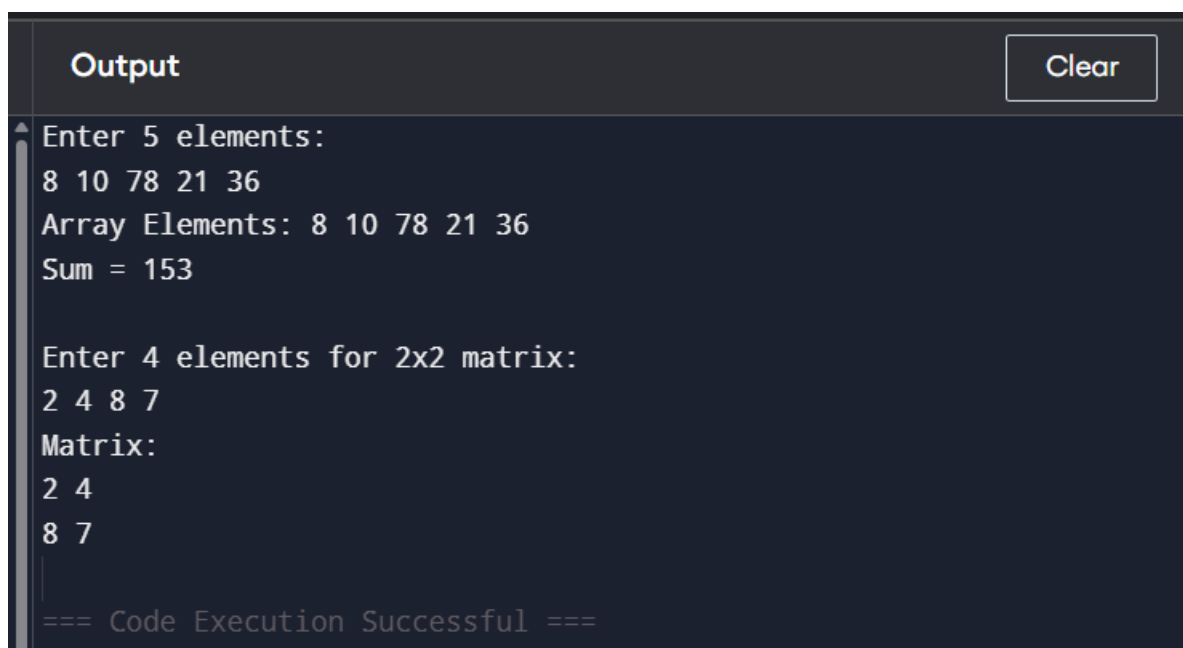
        System.out.print("Array Elements: ");
        for (int i = 0; i < 5; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println("\nSum = " + sum);

        // Two-Dimensional Array
        int[][] matrix = new int[2][2];

        System.out.println("\nEnter 4 elements for 2x2 matrix:");
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }
    }
}
```

```
        System.out.println("Matrix:");
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                System.out.print(matrix[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

❖ OUTPUT



```
Output
```

Enter 5 elements:
8 10 78 21 36
Array Elements: 8 10 78 21 36
Sum = 153

Enter 4 elements for 2x2 matrix:
2 4 8 7
Matrix:
2 4
8 7

=== Code Execution Successful ===

Clear

PRACTICAL-7a

❖ **AIM:** Maximum value and Second Maximum value without duplicates.

❖ **CODE**

```
public class Practical7_a {

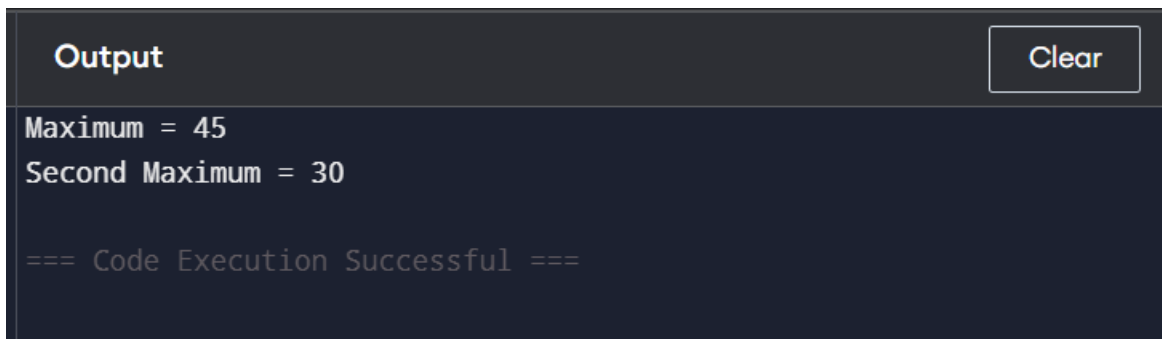
    static void findMaxSecondMax1D(int[] arr) {
        int max = Integer.MIN_VALUE;
        int secondMax = Integer.MIN_VALUE;

        for (int i = 0; i < arr.length; i++) {

            if (arr[i] > max) {
                secondMax = max;
                max = arr[i];
            }
            else if (arr[i] > secondMax && arr[i] != max) {
                secondMax = arr[i];
            }
        }
        System.out.println("Maximum = " + max);
        System.out.println("Second Maximum = " + secondMax);
    }

    public static void main(String[] args) {
        int[] arr1 = {10, 25, 45, 45, 30, 20};
        findMaxSecondMax1D(arr1);
    }
}
```

❖ **OUTPUT**



```
Output
Maximum = 45
Second Maximum = 30

=== Code Execution Successful ===
```

PRACTICAL-7b

❖ **AIM:** Sort the names in Ascending Order.

❖ **CODE:**

```
import java.util.Arrays;

public class Practical7b {
    public static void main(String[] args) {

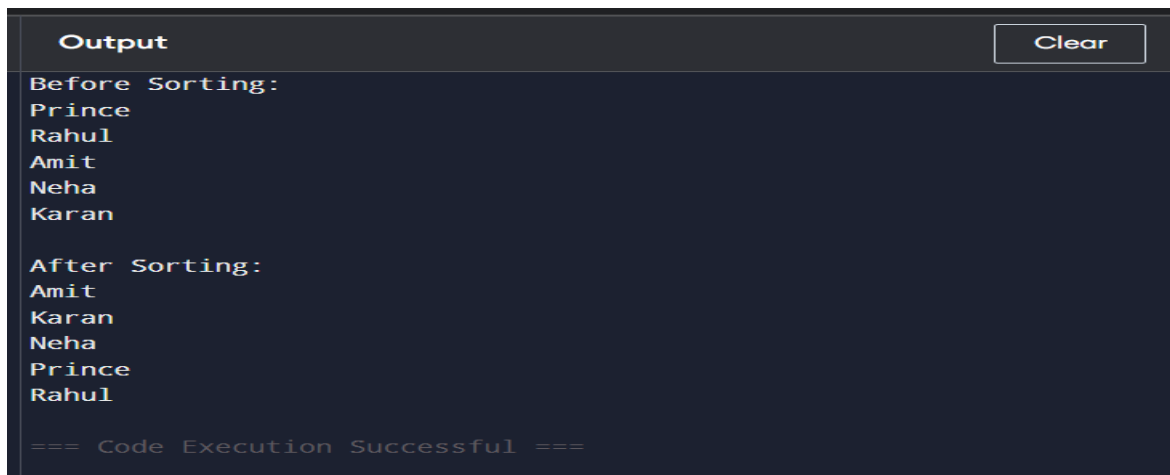
        String[] names = {"Prince", "Rahul", "Amit", "Neha", "Karan"};

        System.out.println("Before Sorting:");
        for (String name : names) {
            System.out.println(name);
        }

        // Sort names in ascending order
        Arrays.sort(names);

        System.out.println("\nAfter Sorting:");
        for (String name : names) {
            System.out.println(name);
        }
    }
}
```

❖ **OUTPUT**



```
Output Clear
Before Sorting:
Prince
Rahul
Amit
Neha
Karan

After Sorting:
Amit
Karan
Neha
Prince
Rahul

=== Code Execution Successful ===
```

PRACTICAL-7c

❖ **AIM:** Addition of two matrix.

❖ **CODE**

```
import java.util.Scanner;
public class Practical7_c {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int[][] A = new int[3][3];
        int[][] B = new int[3][3];
        int[][] C = new int[3][3];

        System.out.println("Enter First Matrix:");

        for (int i = 0; i < 3; i++){
            for (int j = 0; j < 3; j++){
                A[i][j] = sc.nextInt();
            }
        }

        System.out.println("Enter Second Matrix:");

        for (int i = 0; i < 3; i++){
            for (int j = 0; j < 3; j++){
                B[i][j] = sc.nextInt();
            }
        }

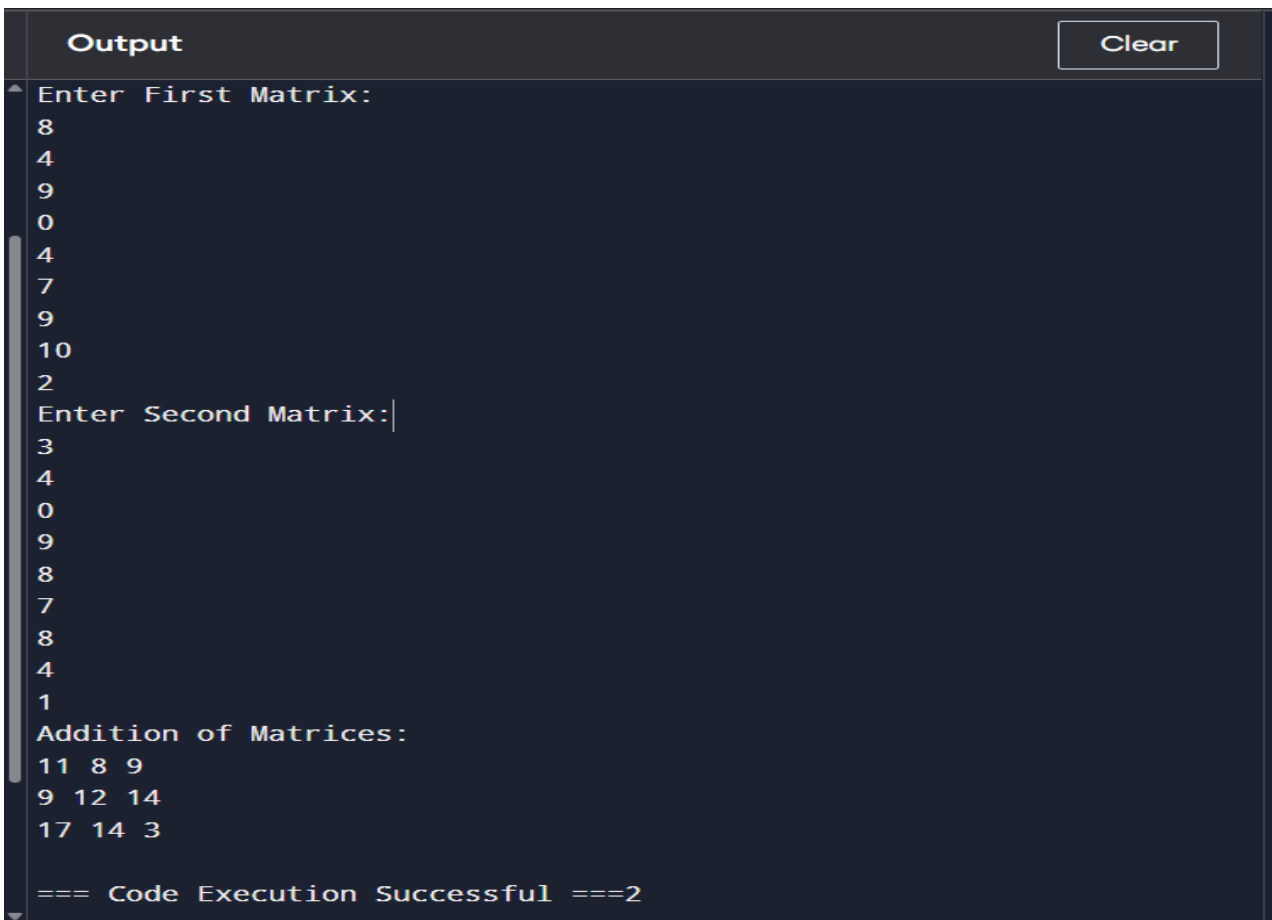
        for (int i = 0; i < 3; i++){
            for (int j = 0; j < 3; j++){
                C[i][j] = A[i][j] + B[i][j];
            }
        }
    }
}
```

```
System.out.println("Addition of Matrices:");

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        System.out.print(C[i][j] + " ");
    }
    System.out.println();
}

sc.close();
}
```

❖ OUTPUT



```
Output
Enter First Matrix:
8
4
9
0
4
7
9
10
2
Enter Second Matrix:
3
4
0
9
8
7
8
4
1
Addition of Matrices:
11 8 9
9 12 14
17 14 3

=== Code Execution Successful ===
```

PRACTICAL-7d

❖ **AIM :** 3x3 Matrix Multiplication.

❖ **CODE**

```
import java.util.Scanner;

public class Practical7_d {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int[][] a = new int[3][3];
        int[][] b = new int[3][3];
        int[][] result = new int[3][3];

        System.out.println("Enter elements of Matrix A:");

        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                a[i][j] = sc.nextInt();
            }
        }

        System.out.println("Enter elements of Matrix B:");

        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                b[i][j] = sc.nextInt();
            }
        }

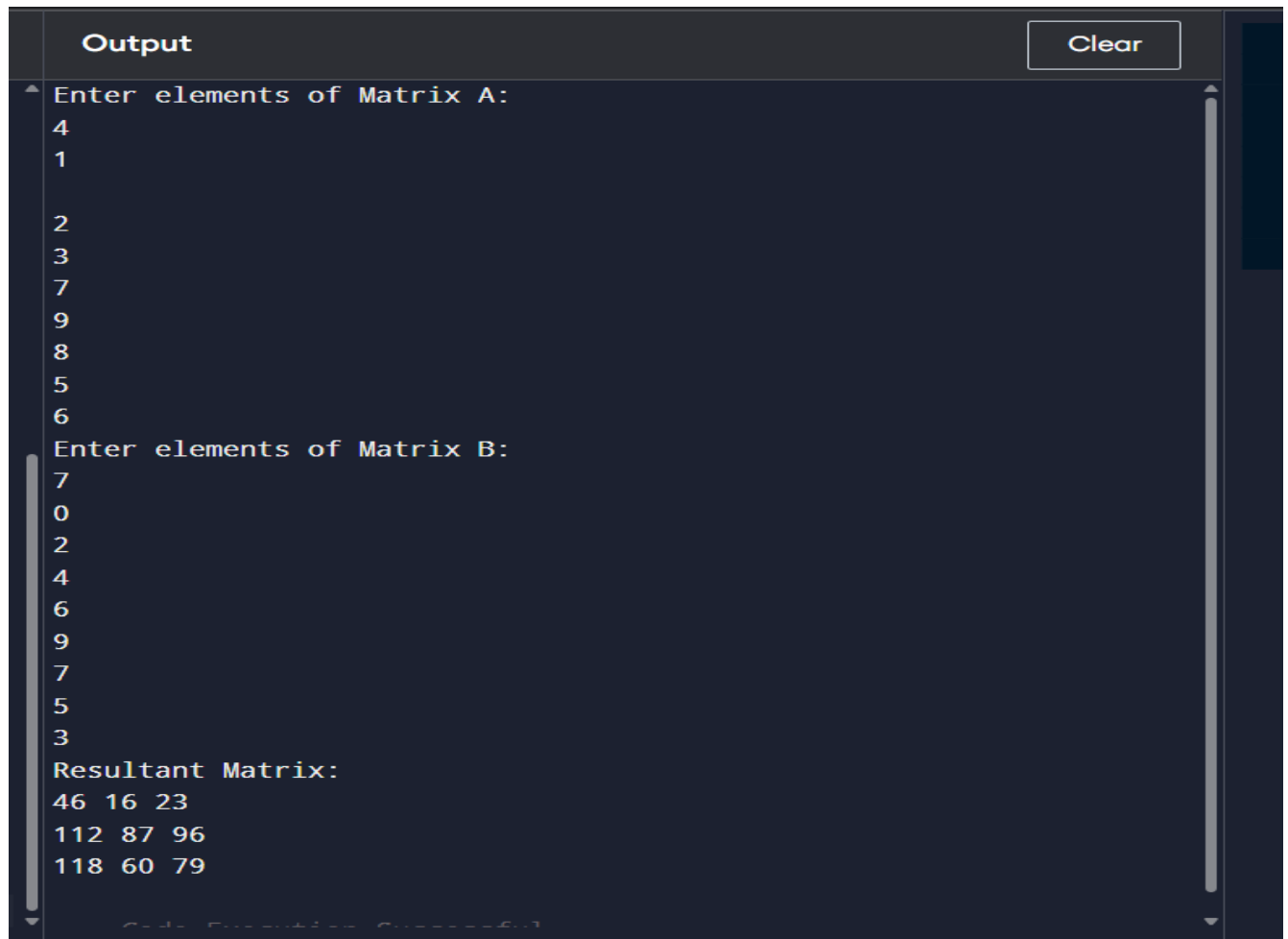
        // Matrix Multiplication
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                result[i][j] = 0;

                for (int k = 0; k < 3; k++) {
                    result[i][j] += a[i][k] * b[k][j];
                }
            }
        }
    }
}
```

```
System.out.println("Resultant Matrix:");

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        System.out.print(result[i][j] + " ");
    }
    System.out.println();
}
}
```

❖ OUTPUT



The screenshot shows a Java IDE output window with a dark background. At the top, there is a tab labeled "Output" and a "Clear" button. The output text is as follows:

```
Enter elements of Matrix A:
4
1
2
3
7
9
8
5
6
Enter elements of Matrix B:
7
0
2
4
6
9
7
5
3
Resultant Matrix:
46 16 23
112 87 96
118 60 79
```

At the bottom of the output window, the text "Code Execution Successful" is visible.

PRACTICAL-8

❖ **AIM:** Write a Java program to demonstrate string handling using String, StringBuffer, and StringBuilder.

❖ **CODE:**

```
import java.util.Scanner;

public class Practical8 {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // ----- String -----
        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        System.out.print("Enter another string: ");
        String str2 = sc.nextLine();

        System.out.println("\n--- String ---");
        System.out.println("Original String: " + str);
        System.out.println("Length: " + str.length());
        System.out.println("Uppercase: " + str.toUpperCase());
        System.out.println("Concatenation: " + str.concat(" " + str2));

        // ----- StringBuffer -----
        StringBuffer sb = new StringBuffer(str);

        System.out.println("\n--- StringBuffer ---");
        System.out.println("Original StringBuffer: " + sb);

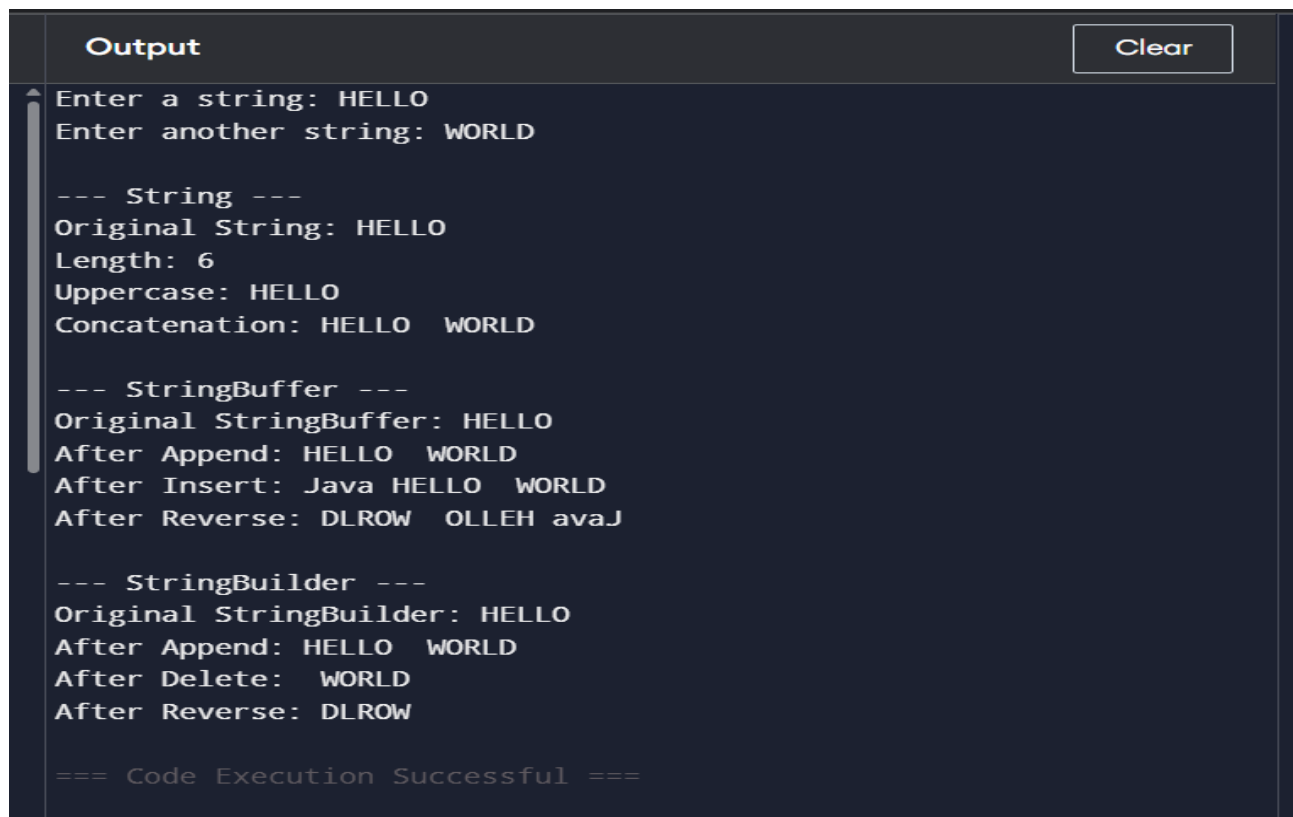
        sb.append(" ").append(str2);
        System.out.println("After Append: " + sb);

        sb.insert(0, "Java ");
        System.out.println("After Insert: " + sb);

        sb.reverse();
        System.out.println("After Reverse: " + sb);
    }
}
```

```
// ----- StringBuilder -----  
StringBuilder sbuilder = new StringBuilder(str);  
  
System.out.println("\n--- StringBuilder ---");  
System.out.println("Original StringBuilder: " + sbuilder);  
  
sbuilder.append(" ").append(str2);  
System.out.println("After Append: " + sbuilder);  
  
sbuilder.delete(0, str.length());  
System.out.println("After Delete: " + sbuilder);  
  
sbuilder.reverse();  
System.out.println("After Reverse: " + sbuilder);  
  
sc.close();  
    }  
}
```

❖ OUTPUT



```
Output  
Enter a string: HELLO  
Enter another string: WORLD  
  
--- String ---  
Original String: HELLO  
Length: 6  
Uppercase: HELLO  
Concatenation: HELLO  WORLD  
  
--- StringBuffer ---  
Original StringBuffer: HELLO  
After Append: HELLO  WORLD  
After Insert: Java HELLO  WORLD  
After Reverse: DLROW  OLLEH avaJ  
  
--- StringBuilder ---  
Original StringBuilder: HELLO  
After Append: HELLO  WORLD  
After Delete:  WORLD  
After Reverse: DLROW  
  
=== Code Execution Successful ===
```


PRACTICAL-9

❖ **AIM :** Write a Java program to check the word is palindrome or not.

❖ **CODE**

```
import java.util.Scanner;

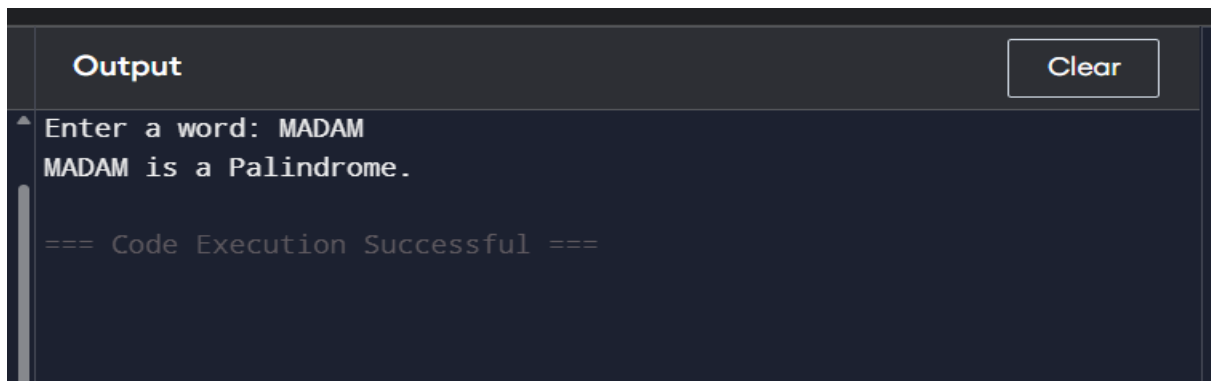
public class Practical9 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a word: ");
        String word = sc.nextLine();
        String reverse = "";

        // Reverse the word
        for (int i = word.length() - 1; i >= 0; i--) {
            reverse = reverse + word.charAt(i);
        }

        // Check palindrome
        if (word.equalsIgnoreCase(reverse)) {
            System.out.println(word + " is a Palindrome.");
        } else {
            System.out.println(word + " is not a Palindrome.");
        }
        sc.close();
    }
}
```

❖ **OUTPUT**



The screenshot shows a dark-themed IDE output window. At the top, the word "Output" is displayed in white, and a "Clear" button is on the right. Below, the program's execution is shown in a monospaced font. It starts with the prompt "Enter a word: MADAM", followed by the output "MADAM is a Palindrome.". At the bottom, a green status message reads "=== Code Execution Successful ===".

PRACTICAL-10

❖ **AIM:** Write a Java program to create a class and object and demonstrate constructors.

❖ **CODE**

```
class Student {  
  
    int rollNo;  
    String name;  
    double marks;  
    // Default Constructor  
    Student() {  
        rollNo = 101;  
        name = "Unknown";  
        marks = 0.0;  
    }  
    // Parameterized Constructor  
    Student(int r, String n, double m) {  
        rollNo = r;  
        name = n;  
        marks = m;  
    }  
    // Method to display student details  
    void display() {  
        System.out.println("Roll No: " + rollNo);  
        System.out.println("Name: " + name);  
        System.out.println("Marks: " + marks);  
    }  
}  
public class Practical10 {  
  
    public static void main(String[] args) {  
  
        // Object using Default Constructor  
        Student s1 = new Student();  
  
        System.out.println("Student 1 - Default Constructor");  
        s1.display();  
        System.out.println();  
    }  
}
```

// Object using Parameterized Constructor

```
Student s2 = new Student(102, "Rahul", 85.5);
```

```
System.out.println("Student 2 - Parameterized Constructor");  
s2.display();
```

```
    }  
}
```

❖ OUTPUT

```
Student 1 - Default Constructor
```

```
Roll No: 101
```

```
Name: Unknown
```

```
Marks: 0.0
```

```
Student 2 - Parameterized Constructor
```

```
Roll No: 102
```

```
Name: Rahul
```

```
Marks: 85.5
```

PRACTICAL-11

❖ **AIM:** Write a Java program to demonstrate encapsulation using access specifiers.

❖ **CODE**

```
class Student {  
    // Private data members  
  
    private int rollNo;  
    private String name;  
    private double marks;  
  
    // Public setter methods  
    public void setRollNo(int rollNo) {  
        this.rollNo = rollNo;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public void setMarks(double marks) {  
        this.marks = marks;  
    }  
  
    // Public getter methods  
    public int getRollNo() {  
        return rollNo;  
    }  
    public String getName() {  
        return name;  
    }  
    public double getMarks() {  
        return marks;  
    }  
}  
  
public class Practical11 {  
    public static void main(String[] args) {  
  
        // Creating object  
        Student s = new Student();
```

```
    // Setting values using setter methods
    s.setRollNo(101);
    s.setName("Rahul");
    s.setMarks(85.5);

    // Getting values using getter methods
    System.out.println("Student Details");
    System.out.println("Roll No: " + s.getRollNo());
    System.out.println("Name: " + s.getName());
    System.out.println("Marks: " + s.getMarks());
}
}
```

❖ OUTPUT

```
Student Details

Roll No: 101

Name: Rahul

Marks: 85.5
```

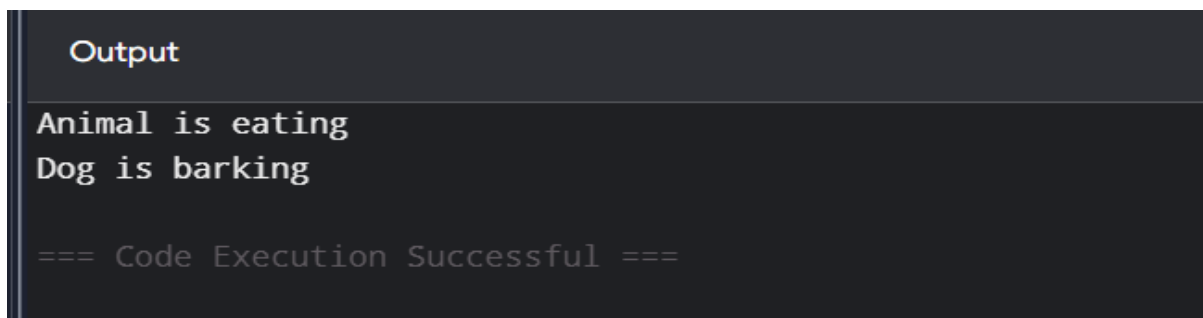
PRACTICAL-12a

❖ **AIM:** Write a Java program to demonstrate inheritance: a) Single Inheritance.

❖ **CODE**

```
class Animal {  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog is barking");  
    }  
}  
  
public class Practical12_a {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
  
        d.eat(); // Parent class method  
        d.bark(); // Child class method  
    }  
}
```

❖ **OUTPUT**



```
Output  
  
Animal is eating  
Dog is barking  
  
=== Code Execution Successful ===
```

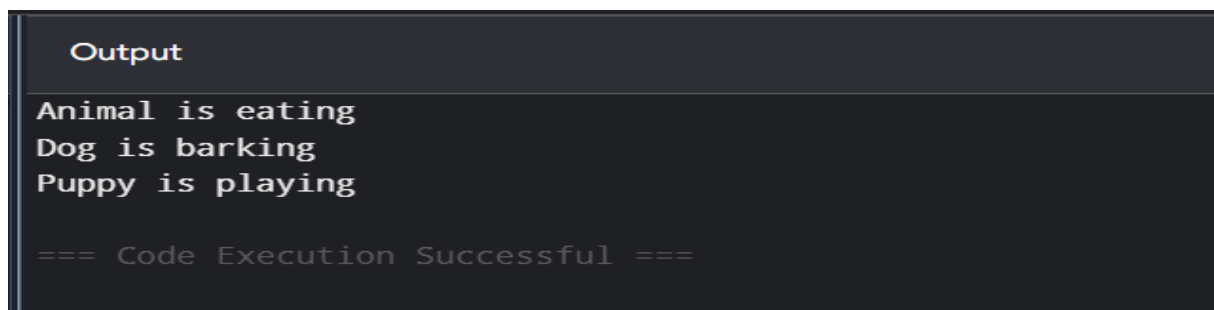
PRACTICAL-12b

❖ **AIM:** Write a Java program to demonstrate inheritance: b) Multilevel Inheritance.

❖ **CODE**

```
class Animal {  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog is barking");  
    }  
}  
class Puppy extends Dog {  
    void play() {  
        System.out.println("Puppy is playing");  
    }  
}  
  
public class Practical12_b {  
    public static void main(String[] args) {  
        Puppy p = new Puppy();  
  
        p.eat(); // Animal  
        p.bark(); // Dog  
        p.play(); // Puppy  
    }  
}
```

❖ **OUTPUT**



```
Output  
Animal is eating  
Dog is barking  
Puppy is playing  
  
=== Code Execution Successful ===
```

PRACTICAL-12c

❖ **AIM:** Write a Java program to demonstrate inheritance: c) Hierarchical Inheritance.

❖ **CODE**

```
class Animal {  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog is barking");  
    }  
}  
  
class Cat extends Animal {  
    void meow() {  
        System.out.println("Cat is meowing");  
    }  
}  
  
public class Practical12_c {  
    public static void main(String[] args) {  
  
        Dog d = new Dog();  
        d.eat();  
        d.bark();  
  
        Cat c = new Cat();  
        c.eat();  
        c.meow();  
    }  
}
```


❖ OUTPUT

```
Output  
  
Animal is eating  
Dog is barking  
Animal is eating  
Cat is meowing  
  
=== Code Execution Successful ===
```

PRACTICAL-12d

❖ **AIM:** Write a Java program to demonstrate inheritance: d) Hybrid Inheritance.

❖ **CODE**

```
class Animal {  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}  
  
interface Pet {  
    void play();  
}  
  
interface Guard {  
    void protect();  
}  
  
class Dog extends Animal implements Pet, Guard {  
  
    public void play() {  
        System.out.println("Dog is playing");  
    }  
  
    public void protect() {  
        System.out.println("Dog is protecting");  
    }  
  
    void bark() {  
        System.out.println("Dog is barking");  
    }  
}
```

```
public class Practical12_d {  
    public static void main(String[] args) {  
  
        Dog d = new Dog();  
  
        d.eat();    // From Animal  
        d.bark();   // Dog's own method  
        d.play();   // From Pet interface  
        d.protect(); // From Guard interface  
    }  
}
```

❖ OUTPUT

```
Output  
Animal is eating  
Dog is barking  
Dog is playing  
Dog is protecting  
  
=== Code Execution Successful ===
```

PRACTICAL-13

❖ **AIM:** Write a Java program to demonstrate use of the this and super keywords.

❖ **CODE**

```
class Parent {
    int number = 100;

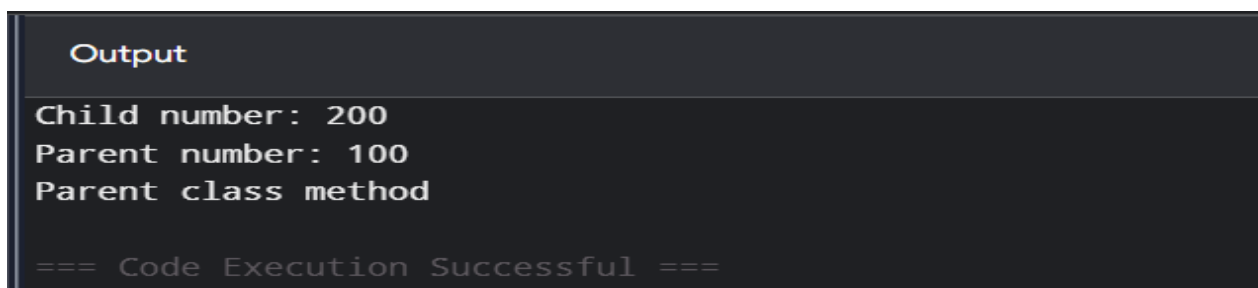
    void display() {
        System.out.println("Parent class method");
    }
}
class Child extends Parent {
    int number = 200;

    void show() {
        // this refers to current class object
        System.out.println("Child number: " + this.number);

        // super refers to parent class variable
        System.out.println("Parent number: " + super.number);

        // Calling parent class method using super
        super.display();
    }
}
public class Practical13 {
    public static void main(String[] args) {
        Child c = new Child();
        c.show();
    }
}
```

❖ **OUTPUT**

The screenshot shows a dark-themed terminal window. At the top, the word "Output" is displayed in a light blue font. Below it, the program's output is shown in a light green monospaced font: "Child number: 200", "Parent number: 100", and "Parent class method". At the bottom of the terminal, the text "=== Code Execution Successful ===" is displayed in a light blue font.

```
Output
Child number: 200
Parent number: 100
Parent class method

=== Code Execution Successful ===
```

PRACTICAL-14

❖ **AIM:** Write a Java program to demonstrate polymorphism (method overloading and overriding).

❖ **CODE**

```
class Animal {  
  
    // Method Overloading  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
  
    void sound(String type) {  
        System.out.println("Animal sound: " + type);  
    }  
}  
  
// Method Overriding  
class Dog extends Animal {  
  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
public class Practical14 {  
    public static void main(String[] args) {  
  
        // Method Overloading  
        Animal a = new Animal();  
        a.sound();  
        a.sound("Roar");  
  
        // Method Overriding  
        Animal d = new Dog();  
        d.sound();  
    }  
}
```

❖ OUTPUT

Output

```
Animal makes a sound
```

```
Animal sound: Roar
```

```
Dog barks
```

```
=== Code Execution Successful ===
```

PRACTICAL-15

❖ **AIM:** Write a Java program to implement abstraction using abstract classes and interfaces(Multiple Inheritance).

❖ **CODE**

```
abstract class Animal {  
  
    // Abstract method  
    abstract void sound();  
  
    // Concrete method  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}  
  
// First interface  
interface Pet {  
    void play();  
}  
// Second interface  
interface Guard {  
    void protect();  
}  
// Multiple inheritance using interfaces  
class Dog extends Animal implements Pet, Guard {  
  
    // Implementing abstract class method  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
  
    // Implementing Pet interface method  
    @Override  
    public void play() {  
        System.out.println("Dog is playing");  
    }  
}
```

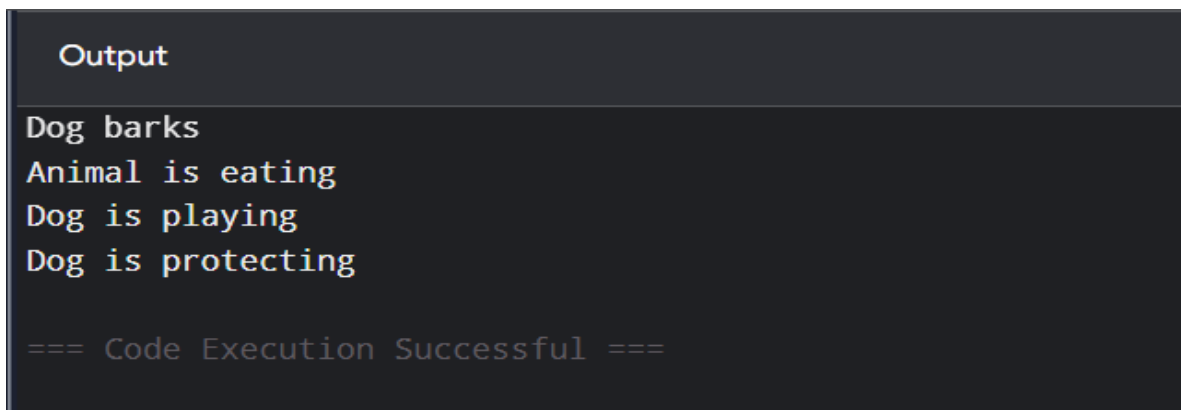
```
// Implementing Guard interface method
@Override
public void protect() {
    System.out.println("Dog is protecting");
}
}

public class AbstractionDemo {
    public static void main(String[] args) {

        Dog d = new Dog();

        d.sound();
        d.eat();
        d.play();
        d.protect();
    }
}
```

❖ OUTPUT

The screenshot shows a dark-themed window titled "Output". It contains the following text: "Dog barks", "Animal is eating", "Dog is playing", and "Dog is protecting", each on a new line. At the bottom, it says "=== Code Execution Successful ===".

```
Output

Dog barks
Animal is eating
Dog is playing
Dog is protecting

=== Code Execution Successful ===
```

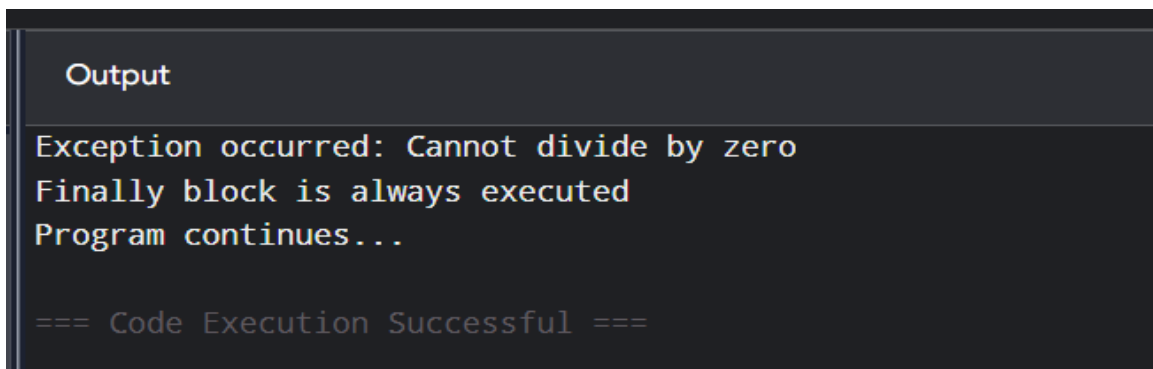

PRACTICAL-16

❖ **AIM:** Write a Java program to demonstrate exception handling using try-catch-finally blocks.

❖ **CODE**

```
public class Practical16{  
  
    public static void main(String[] args) {  
  
        try {  
            int a = 10;  
            int b = 0;  
  
            int result = a / b;  
  
            System.out.println("Result: " + result);  
        }  
  
        catch (ArithmeticException e) {  
            System.out.println("Exception occurred: Cannot divide by zero");  
        }  
  
        finally {  
            System.out.println("Finally block is always executed");  
        }  
  
        System.out.println("Program continues...");  
    }  
}
```

❖ **OUTPUT**



```
Output  
Exception occurred: Cannot divide by zero  
Finally block is always executed  
Program continues...  
=== Code Execution Successful ===
```

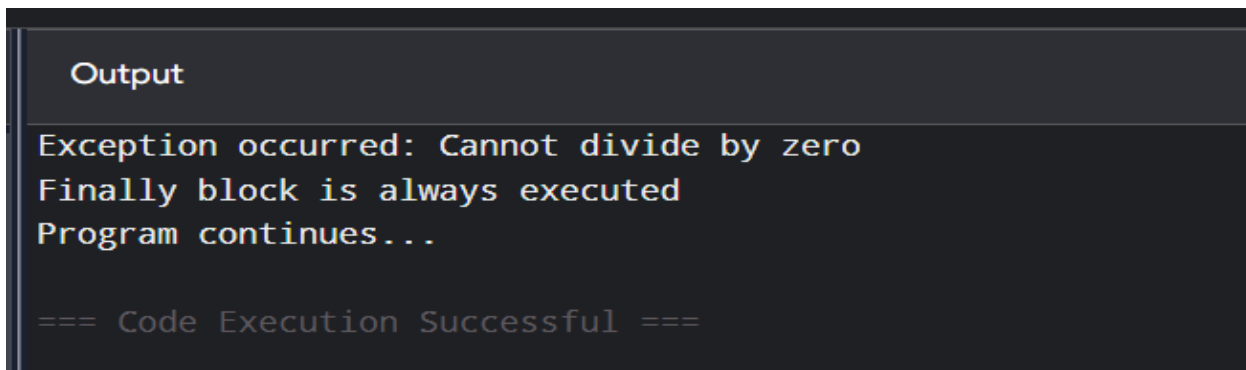
PRACTICAL-17

❖ **AIM:** Write a Java program to demonstrate exception handling using try-catch-finally blocks.

❖ **CODE**

```
public class Practical17{  
  
    public static void main(String[] args) {  
  
        try {  
            int a = 10;  
            int b = 0;  
  
            int result = a / b;  
  
            System.out.println("Result: " + result);  
        }  
  
        catch (ArithmeticException e) {  
            System.out.println("Exception occurred: Cannot divide by zero");  
        }  
  
        finally {  
            System.out.println("Finally block is always executed");  
        }  
  
        System.out.println("Program continues...");  
    }  
}
```

❖ **OUTPUT**



```
Output  
Exception occurred: Cannot divide by zero  
Finally block is always executed  
Program continues...  
=== Code Execution Successful ===
```

PRACTICAL-18

❖ **AIM:** Write a Java program to implement List interface using ArrayList and LinkedList.

❖ **CODE**

```
import java.util.*;

public class Practical18 {

    public static void main(String[] args) {

        // ArrayList
        List<String> arrayList = new ArrayList<>();

        arrayList.add("Java");
        arrayList.add("Python");
        arrayList.add("C++");

        System.out.println("ArrayList: " + arrayList);

        // LinkedList
        List<String> linkedList = new LinkedList<>();

        linkedList.add("HTML");
        linkedList.add("CSS");
        linkedList.add("JavaScript");

        System.out.println("LinkedList: " + linkedList);

        // Common List operations
        arrayList.add("SQL");
        arrayList.remove("C++");

        System.out.println("Updated ArrayList: " + arrayList);
        System.out.println("First element: " + linkedList.get(0));
        System.out.println("Size of LinkedList: " + linkedList.size());
    }
}
```

❖ OUTPUT

```
Output

ArrayList: [Java, Python, C++]
LinkedList: [HTML, CSS, JavaScript]
Updated ArrayList: [Java, Python, SQL]
First element: HTML
Size of LinkedList: 3

=== Code Execution Successful ===
```

PRACTICAL-19

❖ **AIM:** Write a Java program to implement Set interface using HashSet and TreeSet.

❖ **CODE**

```
import java.util.*;

public class SetDemo {

    public static void main(String[] args) {

        // HashSet
        Set<Integer> hashSet = new HashSet<>();

        hashSet.add(30);
        hashSet.add(10);
        hashSet.add(20);
        hashSet.add(10); // Duplicate element

        System.out.println("HashSet: " + hashSet);

        // TreeSet
        Set<Integer> treeSet = new TreeSet<>();

        treeSet.add(30);
        treeSet.add(10);
        treeSet.add(20);
        treeSet.add(10); // Duplicate element

        System.out.println("TreeSet: " + treeSet);

        // Common Set operations
        hashSet.remove(20);

        System.out.println("Updated HashSet: " + hashSet);
        System.out.println("Does TreeSet contain 20? " + treeSet.contains(20));
        System.out.println("Size of TreeSet: " + treeSet.size());
    }
}
```

❖ OUTPUT

Output

```
HashSet: [20, 10, 30]  
TreeSet: [10, 20, 30]  
Updated HashSet: [10, 30]  
Does TreeSet contain 20? true  
Size of TreeSet: 3  
  
=== Code Execution Successful ===
```

PRACTICAL-20

❖ **AIM:** Write a Java program to implement Set interface using HashSet and TreeSet.

❖ **CODE**

```
import java.util.*;
public class Practical20 {
    public static void main(String[] args) {

        // PriorityQueue
        Queue<Integer> priorityQueue = new PriorityQueue<>();

        priorityQueue.add(30);
        priorityQueue.add(10);
        priorityQueue.add(20);

        System.out.println("PriorityQueue: " + priorityQueue);
        System.out.println("Removed element: " + priorityQueue.poll());
        System.out.println("After removal: " + priorityQueue);

        // Deque
        Deque<Integer> deque = new ArrayDeque<>();
        deque.add(10);
        deque.add(20);
        deque.add(30);

        System.out.println("\nDeque: " + deque);

        // Add elements at both ends
        deque.addFirst(5);
        deque.addLast(40);

        System.out.println("After adding at both ends: " + deque);

        // Remove elements from both ends
        System.out.println("Removed first: " + deque.removeFirst());
        System.out.println("Removed last: " + deque.removeLast());

        System.out.println("After removal: " + deque);
    }
}
```

❖ OUTPUT

```
Output

PriorityQueue: [10, 30, 20]
Removed element: 10
After removal: [20, 30]

Deque: [10, 20, 30]
After adding at both ends: [5, 10, 20, 30, 40]
Removed first: 5
Removed last: 40
After removal: [10, 20, 30]

== Code Execution Successful ==
```


PRACTICAL-21

❖ **AIM:** Write a Java program to implement map interface using HashMap and LinkedHashMap.

❖ **CODE**

```
import java.util.*;
public class Practical21 {

    public static void main(String[] args) {

        // HashMap
        Map<Integer, String> hashMap = new HashMap<>();

        hashMap.put(101, "Java");
        hashMap.put(102, "Python");
        hashMap.put(103, "C++");
        hashMap.put(101, "Java Programming"); // Updates value

        System.out.println("HashMap: " + hashMap);

        // LinkedHashMap
        Map<Integer, String> linkedHashMap = new LinkedHashMap<>();

        linkedHashMap.put(101, "Java");
        linkedHashMap.put(102, "Python");
        linkedHashMap.put(103, "C++");

        System.out.println("LinkedHashMap: " + linkedHashMap);

        // Common Map operations
        System.out.println("Value for key 102: " + hashMap.get(102));

        hashMap.remove(103);

        System.out.println("Updated HashMap: " + hashMap);

        System.out.println("Does LinkedHashMap contain key 101? "
+ linkedHashMap.containsKey(101));
    }
}
```

❖ OUTPUT

Output

```
HashMap: {101=Java Programming, 102=Python, 103=C++}  
LinkedHashMap: {101=Java, 102=Python, 103=C++}  
Value for key 102: Python  
Updated HashMap: {101=Java Programming, 102=Python}  
Does LinkedHashMap contain key 101? true  
  
=== Code Execution Successful ===
```

PRACTICAL-22

❖ **AIM:** Write a Java program to demonstrate multithreading using Thread class and Runnable interface.

❖ **CODE**

```
class MyThread extends Thread {

    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Thread Class: " + i);

            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }

}

class MyRunnable implements Runnable {

    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Runnable Interface: " + i);

            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }

}

public class Practical22 {
```

```
public static void main(String[] args) {  
  
    // Creating thread using Thread class  
    MyThread t1 = new MyThread();  
  
    // Creating thread using Runnable interface  
    MyRunnable obj = new MyRunnable();  
    Thread t2 = new Thread(obj);  
  
    // Starting both threads  
    t1.start();  
    t2.start();  
}  
}
```

❖ OUTPUT

```
.java } ; if ($?) { java Practical22 }  
Runnable Interface: 1  
Thread Class: 1  
Thread Class: 2  
Runnable Interface: 2  
Thread Class: 3  
Runnable Interface: 3  
Runnable Interface: 4  
Thread Class: 4  
Thread Class: 5  
Runnable Interface: 5  
PS C:\Users\princ\OneDrive\Documents\Desktop\Parul 3rd Sem\IT OOPJ>
```